Testes automatizados com Pytest



Testes automatizados

- **Testes automatizados:** trechos de código criados para verificar se outro código está funcionando conforme o esperado
  - Não executa manualmente uma aplicação,
    - Envia uma requisição e confere o resultado visualmente
    - Teste executa essa verificação automaticamente
- **Arrange, Act, Assert:**
  - **Arrange:** preparação do cenário
  - **Act:** execução da funcionalidade
  - **Assert:** verificação do resultado
- **Aplicação:** verificações repetidas sempre que o código for modificado
  - se uma função retorna o valor esperado;
  - se uma API responde com o status HTTP correto;
  - se uma autenticação funciona;
  - se um usuário sem permissão é bloqueado;
  - se dados inválidos são rejeitados;
  - se um rate limit realmente impede excesso de requisições.



Testes automatizados

- **Exemplo simples:**
  - **Função a ser testada:**
    - ```
      def somar(a, b):
          return a + b
      ```
  - **Teste para a função:**
    - ```
      def test_somar():
          resultado = somar(2, 3)
          assert resultado == 5
      ```
      - `assert` verifica se o resultado obtido corresponde ao esperado



Pytest

- **pytest:** framework de testes para Python
  - Escrita de testes usando funções simples
  - Execução automática de todos os testes encontrados no projeto
- **Convenções:**
  - **Arquivo de teste:** começa com **`test_ ou_test.py`**
  - **Funções de teste:** começam com **`test_`**
- **Instalação:** **`pip install pytest`**
- **Execução:** **`pytest`**
- **Diretório de testes:** geralmente, **`tests`** na raiz do projeto
  - **Exemplo:**

```
projeto/
├── main.py
├── models/
├── routes/
│
└── tests/
    └── test_api.py
```

Pytest

- **Assert:** instrução do Python utilizada para verificar se determinada condição é verdadeira
  - Se a condição for verdadeira, o teste passa
- **Sintaxe:** `assert condicao`
- **Exemplos assert:**
  - `assert resultado == 10, assert usuario is not None`
  - `assert response.status_code == 200`
  - `assert "access_token" in response.json()`
- **Exemplo de teste** `test_api.py`**:**
  - ```
    def test_exemplo():
        assert 1 + 1 == 2
    ```


    ```

Testes de API

- **Testes de API:** verifica o comportamento de endpoints HTTP
  - método HTTP; URL; dados enviados; headers; status HTTP; corpo da resposta, etc
- **Métodos HTTP do `TestClient`:**
  - `client.get()`
  - `client.post()`
  - `client.put()`
  - `client.delete()`
- **TestClient:** classe do FastAPI para testes
  - **Importação de `TestClient`:**
    - `from fastapi.testclient import TestClient`
  - **Importação da aplicação FastAPI:**
    - `from main import app`
  - **Instanciação do cliente de teste:**
    - `client = TestClient(app)`



Testes de API

- **Exemplo de teste para raiz:**
  - ```
    from fastapi.testclient import TestClient
    from main import app
    client = TestClient(app)
    def test_home():
        response = client.get("/")
        assert response.status_code == 200
        assert response.json() == {
            "message": "ok"
        }
    ```


    ```

CI/CD e Github Actions



GitHub Actions e CI/CD

- **GitHub Actions:** ferramenta de automação integrada ao GitHub
  - Executa tarefas automaticamente quando algum evento acontece no repositório
  - **Exemplo:**
    - Ocorre um push
      - Verifica se aplicação continua funcionando
      - Executa testes
- **CI (Continuous Integration):** testar automaticamente novas alterações no código
  - Exemplo:
    - Desenvolvedor altera código e git push
    - GitHub Actions executa pytest e PASS ou FAIL
- **CD (Continuous Delivery/Deployment):** entrega ou publicação automática da aplicação
  - **Exemplo:**
    - Testes passam
    - Aplicação publicada



GitHub Actions e CI/CD

- **Por que CI/CD nos testes de segurança?**
  - **Sem automação:**
    - Alterar código -> Lembrar de executar pytest->Enviar código
  - **Com automação:**
    - git push->pytest executado automaticamente->PASS / FAIL
- **Github Actions: *Proporciona automação***
  - Alterações podem ser detectadas automaticamente:
    - autenticação
    - autorização
    - validação
    - rate limiting
    - etc
  - **Ideia:**
    - **Trocar:** `pytest tests/ -v`
    - **Por:** Github actions



Workflow

- **Workflow:** arquivo YAML que descreve uma automação
- **Local:**
  - `.github/`
    `└── workflows/`
    `└── tests.yaml`
- **Elementos Principais:**
  - `name:` nome do workflow
  - `on` : quando será executado
  - `jobs:` tarefas que serão executadas
- **Exemplo `tests.yaml`:**

```yaml
name: Testes
on:
  push:
jobs:
  pytest:
    runs-on: ubuntu-latest
    steps:
      - name: Executar teste
        run: echo "Workflow funcionando!"
```

O GitHub cria temporariamente uma máquina Linux para executar o pipeline

Steps

- **Steps:** Passos a serem executados por um job
  - Dentro de um job existem vários steps
- **Exemplo arquivo seção steps no `.yaml`:**

```
- name: Baixar projeto
  uses: actions/checkout@v4

- name: Diagnóstico Repositório
  run: |
    pwd
- name: Instalar dependências
  run: pip install -r requirements.txt

- name: Executar testes
  run: python -m pytest  -v
```

actions: referencia a action
checkout: disponibiliza o código
@v4: versão da Action

